Chapter 8- Exercises

1) a) Yes. Independent Set $\equiv_P$ Vertex Cover, and Interval Scheduling $\leq_P$ Independent Set because we for an instance of Interval Scheduling, we can create a graph G by creating a vertex for each interval an edge b/w vertices iff the coresponding intervals overlap. Then solving this instance of Independent Set in polynomial time solves Interval Scheduling in polynomial time, so Interval Scheduling $\leq_P$ Independent Set $\equiv_P$ Vertex Cover i.e. Interval Scheduling $\leq_P$ Vertex Cover.

b) Unknown. We already have a polynomial time greedy algorithm for Interval Scheduling, so if Indepent Set $\leq_P$ Interval Scheduling then P=NP, which is unknown.

NOTE for part (a) :- We could also just solve Interval Scheduling in polynomial time. If yes, output a fixed 'yes' instance of Vertex Cover, else a fixed 'no' instance.

b) Unknown. If Ind. Set $\leq_P$ Int. Sched then P=NP because Ind. Set is NP complete. If P=NP, then Ind. Set

$\leq_P$ Int. Sched. Thus answering this would resolve the unknown question of whether P=NP.

4)a) Consider any instance of Set Packing (SP). We create an instance of Resource Reservation (RR) as follows:- ~~For the complete set universal set U~~ Create ~~a set of~~ |U| resources, where each resource coresponds to an element of U (the universal set in SP). For each subset S_i in the collection of subsets S in the SP instance, create a process such that the process requests the resources ~~oss~~ coresponding to the elements of S_i. ~~Solving this~~

~~Sup~~ If we solve this RR instance, then ~~either~~ ~~every~~ each process either gets all processes it requests, or none of them, and since resources cannot be shared, ~~each~~ the resources that got their requests fulfilled, that

If it is possible to allocate resources so that at least k processes are active, then ~~each~~ ~~if we~~ take since resources cannot be shared, the ~~resource~~ resources allocated to ~~a~~ the processes ~~is~~ represent pairwise disjoint subsets, each coresponding

to an $S_i \in S$, and there are at least k of them, thus at least k disjoint subsets exist in S. Conversely, if at least k disjoint subsets exist in S, then the processes ~~requesting the asso core~~ with requests coresponding to those subsets do not share any of the resources in their requests, so at least k processes can be activated.

Thus Set Packing $\leq_p$ Resource Reservation and so RR is NP ~~complete~~. Also, we can check a solution to RR by simply checking that in polynomial time that there ~~process~~ are k processes in the certificate solution and none of their requests ask for the same resource, so ~~Thus~~ RR is in NP. ~~complete~~. Hence RR is NP complete.

b) ~~Sudo Let A Take A~~ Iterate over the ~~set~~ processes, for each process compare it to every other process to see if their resource requests ~~do not~~ overlap. ~~If they do, for any pair, we then~~ In this way, we check every possible 2-combination ~~pair~~ of processes. If any of them ~~we~~ do not overlap, return true, else return false. Assuming each process has its

set of requests in a ~~has~~ hashmap set, this algorithm runs in $O(nR)$ time where R is the sum of ~~all~~ the number of resources requested by each process, because there are n iterations and each one compares a ~~se~~ request set with ~~every other request s~~ every other request set (except the ones it ~~checked in~~ already compared with in earlier iterations)

c) This is simply bipartite matching problem. Let P be the set of vertices representing the people resource in each process request, E the set of vertices representing the equipment resource in each process request. We create an directed edge (P_i, E_j) iff there exists a process that requests ~~P~~ person i and equipment j. Now we find a bipartite matching ~~by ea in by adding a S in~~ $O($ by adding a super source to with an edge to every P_i in P and a super sink with an edge from every $E_i \in E$, ~~Every edge has~~ then giving every edge capacity 1 and calculating a max flow.

If a bipartite matching exists with at least k edges, then those edges all are pairwise disjoint (i.e. do not share a node) and ~~so~~ so since ~~k~~ each edge

coresponds to the request of a process, we can activate at least k processes. Conversely if we can activate at least k processes, then that means we have atleast k disjoint subsets, each with a coresponding to an edge in our graph, so we have a bipartite matching of size at least k.

Constructing the graph takes $O(m+n)$ and finding a max flow takes $O(nm)$ using Ford Fulkerson

d) This special case is NP complete. Note that a certifier can check a solution in polynomial time by being provided with the instance and the active processes, and then checking if there are at least k of them, with their resource requests being pairwise disjoint.

Now, suppose we have any instance of independent set problem. We create an equivalent instance of this special case Resource Reservation (SCRR) as in polynomial time follows:- For each node v, we create a process, and that process requests a unique resource for each neighbour u of v, except if

Take an instance of Independent Set. In polynomial time, we can create an equivalent instance of Special Case Resource Reservation (SSRR) as follows:- For each node, look at all of its neighbours. Then create a process for this node that requests a unique resource for each neighbour. Repeat for every node, except for neighbours for whom we have already performed this procedure, we put the same resource in our request set as the resource that neighbour has coresponding to us.

Suppose we can activate at least k processes in our SSRR instance. Then those process requests are all disjoint, so the associated nodes do not have an edge b/w them and so an independent set of size at least k exists. Conversely, if an independent set of size at least k exists, then since the nodes are not connected, by our construction the associated processes have disjoint requests, so at least k processes can be active.

Hence we have shown that SSRR $\in$ NP and Independent Set $\leq_p$ SSRR, thus SSRR is NP Complete.

6) ~~We prove that~~ For any instance of monotone SAT (mSAT) we can take as a certificate the ~~input~~ ~~vertices~~ with few true variables in a satisfying assignment that ~~the described version of~~

6) For any instance of monotone SAT (mSAT), we can take the variables that are set to 1 as a certificate and check in polynomial time that there are at most k variables and a variable belonging to each clause. Thus mSAT $\in$ NP.

Now take any instance of Vertex Cover (VC). ~~We give the vertices~~ In polynomial time, we construct an equivalent mSAT instance as follows:- Give each vertex a unique label x_i. Now for each edge (x_i, x_j), create a clause containing only those two labels as variables. We use the same value of k for the mSAT instance as was given in the VC instance.

(this will be the name of the associated variable)

Suppose a satisfying assignment of at most k variables exist. Then since each clause represents an edge and each variable in the clause one of the endpoints of the edge, we have chosen at most k nodes (the ones with the same labels as the variables set to 1) that form a vertex cover. Conversely, if a vertex cover of size at most k exists, then since these k nodes cover all edges, the associated variables cover all clauses, and so a satisfying mSAT assignment of at most k variables exist.

Thus since $SAVC \leq_p {}_m SAT$ and $mSAT \in NP$, $mSAT$ is NP complete.

8) For an instance of Refrigerator Magnets (RM), a certifier can simply take the instance and a set of strings with multiplicities and check in polynomial time that each string is in the set of strings given in the instance and that each symbol in the collection appears the specified number of times in the set of strings. Thus $RM \in NP$.

(3DM)

Take any instance of 3D matching. We can create an equivalent RM instance in polynomial time as follows:- For each triple (X_i, Y_j, Z_k) in $T \subseteq X \times Y \times Z$ supplied in the 3DM instance, we add the string "$X_i Y_j Z_k$" to our collection of strings, and for each element ~~#~~ ~~#~~ E in $X \cup Y \cup Z$ (which can be X_i or Y_i or Z_i), we add it to our collection of symbols with multiplicity of 1.

Suppose there exists a set of n triples so the each element of $X \cup Z \cup Y$ is in exactly one triple. Then, since each triple is a string in our 3DM instance, that string covers 3 symbols (since each symbol has multiplicity 1), and there are 3n symbols, ~~so n strings~~ each for an element of $X \cup Y \cup Z$, in seperate strings cover all of them while satisfying the multiplicity constraint. Conversely, suppose there is a collection of strings that has each symbol the specified number of times. Then since ~~eve~~ there is one of each symbol, ~~for each~~ every string must be unique, and there must be n string since there are 3n symbols. Each string ~~is~~ corresponds to a triple, and hence there ~~are~~ exist n triples so that each each element of $X \cup Y \cup Z$ is in exactly one.

Since We have shown $3DM \leq_p RM$ and $RM \in NP$, hence RM is NP complete.

10)a) ~~We~~ We A certifier can take an instance of Strategic Advertising (SA) and as certificate, ~~the~~ a set of nodes. ~~that~~ Then in ~~poly~~ polynomial time, it can check that there are at most k nodes in the set and that each P_i has a node in the set. Hence SA $\in$ NP.

Take any instance of the version of monotone ~~SAT~~ SAT which we proved NP complete in ~~qu~~ exercise 6. ~~For each~~ We create an equivalent instance of SA in polynomial time as follows:- for each variable x_i, create a node v_i. For each clause c_i, create the associated set of edges to hence create a path P_i (so clause $(x_1\ x_2\ x_3)$ will result in edges (v_1, v_2), (v_2, v_3)). We keep the same value of k as supplied with the mSAT instance.

Suppose there is a satisfying assignment to mSAT with at most k variables set to 1. That means there ~~are~~ is at least one variable in each clause set to 1, and since each clause directly coresponds to a path, seleeting nodes coresponding to the 1 variable in ~~e~~ each ~~e~~

clause gives a set of at most k nodes such that there is at least one of them on each path. Conversely, suppose there are at most k nodes such that each path contains one of these nodes. Since each path coresponds to a clause, and the node with an ad ~~refer~~ in that path coresponds to a variable we can set to 1 in that clause, there is a satisfying assignment with at most k variables set to 1.

~~Sirtlen~~ Since we showed SA $\in$ NP and mSAT $\leq_p$ SA, we have proven SA is NP complete.

b) Call the blackbox on the entire instance of the problem. ~~Re~~ If it ~~se~~ returns no, then our algorithm terminates and returns no as well. Else, continue as follows. Let $V' \subseteq V$ be the ~~se~~ set of nodes ~~in~~ such that each $v' \in V'$ is in at least one of the paths in the set of provided paths $\{P_i\}$. Select any $v' \in V'$. ~~If~~ Call the black box on the smaller instance of the problem, i.e. with the same graph, but k decremented by 1 and the set of paths should have removed those paths that contained v'. ~~And~~

If the blackbox returns no, move on to another node in V' (eventually it will say yes). If it says yes, then we add v' to the 'answer set'. We repeat this procedure on the smaller instance we just called the blackbox on, until we get to ~~k~~ an instance where the set of paths is empty, at which point just return the nodes in the answer set.

Suppose a set of at most k nodes ~~that~~ such the each path has at least one of them exists. Then the algorithm ~~only~~ on each iteration selects a node that ~~will not~~ ensures the paths it does not cover. Can still be covered ~~by~~ without the total number of nodes exceeding k, ~~so~~ and so we get a set of nodes that fulfill the given constraints. Conversely, if an algorith returns a set of nodes A, then we know A covered every path given because, ~~if~~ the algorithm ~~runs~~ runs until no ~~k~~ paths are left uncovered, and $|A| \leq k$ because there are at most k iterations and each ~~path~~ iteration selects one node. Thus there exists a set of at most k nodes such that each path has at least one

node from the set.

Let $L = \sum |P_i|$. There are at most k iterations and each iteration iterates over at most $|V|$ nodes. Additionally for each node we check for its existence in at most each path P_i, so the algorithms takes $O(|V| \cdot L \cdot k)$ time. The black box is called for each node we iterate over, so there are $O(|V|k)$ i.e a polynomial number of blackbox calls.

19)a) Create a source s, a sink t, a node c_i for each of the n canisters, a node l_i for each of the m ships, Create edge (s, c_i) for each c_i with capacity 1, edge (c_i, l_j) for each truck l_j which can carry c_i (for every c_i) and give it capacity 1, and edge (l_i, t) for each l_i with capacity k. Calculate max flow f. If $v(f) = n$, return true, else return false.

If there is a way to safely load all canisters without overloading the truck, then each canister will be matched to a truck via

a flow saturated edge, and so every source edge will be saturated with flow so $v(f) = n$ and the algorithm returns yes. Conversely if the algorithm returns yes, then $v(f) = n$ so each n was matched to a truck it was compatible with, and due to (t, l) edges having capacity k, the truck were not overloaded, so there is a way to load all n cannisters without overloading the truck.

Creating the flow network takes $O(nm)$ time, and using scaled fold fulkerson to calculate max flow takes time $O(nm\lg(n))$ time so overall runtime is $O(nm\lg(n))$ which is polynomial

b) We first show truck filling (TF) is in NP. For any instance of TF, A certifier can take any instance of TF, and as certificate, the cannisters that go in each truck, and check in polynomial time that each truck has at most k cannisters, there are at most m trucks, and every cannister is in a truck and incompatible cannisters are not in the same truck.

i.e the described problem

truck. Hence TF ∈ NP.

Take any instance of 3-Colouring (3C) (ie G graph $G=(V,E)$). We create an equivalent TF instance in polynomial time as follows:- For each node, v create a cannister C_v. For each edge (u, v), make it so C_v and C_u cannot share a truck. Create 3 trucks (i.e. m=3) and each truck can hold at most n (where n=|V|) cannisters.

Suppose a 3 colouring exists. Then adjacent nodes get seperate colours, and since each pair of adjacent nodes corespond to incompatible cannisters, incompatible cannisters can go into separate trucks. Additionally, there are at most 3 colours used so the cannisters can go into at the 3 trucks, and each truck has capcaity for all cannisters. Conversely, if all |V| cannisters can be loaded into the 3 trucks without incompatible cannisters sharing a truck, then since incompatible cannisters corespond to adjacent nodes, they can be given the colour associated with the truck their coresponding cannisters are in to form a valid three colouring.

Thus we have shown 3-Colouring $\leq_p$

TF and TF $\in$ NP, hence TF is NP complete.

2(a) We first show low diameter clustering problem (LDCP) is in NP. A certifier can take an instance of LDCP as well as k sets and check in poly-time the sets cover all objects and no two items in a set has distance greater than B.

Take any instance of 3 colouring (3C). We create an equivalent LDCP instance in polynomial time as follows:- For each node, ~~create~~ v create an object p_v. Let k=3, B=2, ~~for~~ and for any edge (u,v), we have $dist(p_v, p_u) = 3$, else if two nodes dont share an edge, they have distance 1.

Suppose a ~~3~~ valid 3 colouring exists. Then any two nodes that are adjacent have different colours i.e. are put in the different clusters associated with each colour because their ~~di~~corresponding objects have distance greater than B=3, ~~otherwise~~ and the rest can be put in any cluster hence solving LDCP instance. Conversely suppose ~~we can a valid set of 3~~ 3 valid clusters exist. Then ~~the since each~~ we can give the nodes the colours

cluster
coresponding to the set their objects are in, and since adjacent nodes have objects in seperate clusters, this gives us a valid 3 colouring.

Hence $3C \leq_p LDCP$ and $LDCP \in NP$, thus LDCP is NP complete.

22) Suppose we are given an instance as $G=(V, E)$ and an integer k. First call the blackbox on the full instance. If it returns yes or no, we return the blackbox's answer. Else if it says G is disconnected, continue as follows:-

Find all connected components of G (i.e. $C = \{G_i\}$). For each $G_i \in C$, we set $k_i = 0$. Then we do $|V|+1$ iterations. In each iteration, we call the blackbox with the instance (G_i, k_i) for each G_i, and we increment k_i by 1 before the next iteration. If the blackbox returns no for some G_i, set $k'_i = k_i - 1$, and exclude G_i from future calls to the blackbox. At the end, sum all k'_i for each $G_i \in C$, if the sum $k' \geq k$, return true, else return false.

Suppose our algorithm returns true, then either if G was connected, the black-box solved the answer correctly. Else we found the size of the largest independent set in each connected component, and since connected components are pairwise disjoint, the sum k' is the size of the largest independent set in G, so returning whether $k' \geq k$ correctly tells us whether an independ set of size at least k exists in G. Conversely, suppose an independ set of size at least k exists in G. Then if G is connected, our algorithm correctly returns yes because the blackbox can directly answer. Else if G is disconnected, then the any independent set of size at least k will have at least one node in each connected component, forming an independent set in its own connected component, and the sizes of these sets add up to k and since our algo calculates the size of the largest independent subset possible in each connected component, we get $k' \geq k$ and our algorithm returns true.

Finding all connected components take $O(|V| + |E|)$ time, there are $O(|V|)$ iterations, and each iteration has $O(|C|)$ calls to A, so if $r(A)$ is A's running

time then our algorithm runs in $O(|V| \cdot |C| \cdot r(A))$ time.

23) Suppose $m \leq n$ and let L denote the maximum length of any string in $A \cup B$. Suppose there is a string that is a concatenation over both A and B, and let u be one of minimum length.

We say position p in u is of type (a_i, k) if in the concatenation over A, it is represented by position k of string a_i. We define (b_i, k) analogously. Now if the length of u is greater than $n^2 L^2$ then by the pigeonhole princleiple, there exists positions p and p' in u, $p < p'$ so that both are of type (a_i, k) and (b_j, k) for some indices i, j, k. But in this case, the string u' obtained by deleting positions $p, p+1 \ldots p'-1$ would also be a concatendion over A and B. As u' is shorter than u, this is a contradiction hence $u \leq n^2 L^2$.

26) We start by showing Number Partitioning Problem (NPP) is in NP. A certifier can take as certificate, of the two sets S_1 and S_2, which are polynomial in the size of the NPP instance,

and check that they are disjoint and ~~each~~ that their intersection equals S (the set of all given positive integers in the NPP instance) and that the sum of integers in S_1 and S_2 are the same, all in polynomial time.

Now ~~sup~~ take any instance of SubSet Sum (SS). ~~We~~ ~~If~~ ~~the~~ ~~sum~~ ~~of~~ ~~the~~ we create an equivalent NPP instance in polynomial time as follows: if the total sum of all the weights ~~is les~~ 'SUM' is less than W, create a fixed no instance of NPP. If SUM = W, create a fixed yes instance. If SUM = 2W, then for each w_i, simply create x_i such that $x_i = w_i$. If W < SUM < 2W, then for each ~~$x_i \in w_i$~~, create x_i such that $w_i = x_i$, and add one final $x_{n+1} = 2W - SUM$. Else if 2W < SUM, then make the final $x_{n+1} = SUM - 2W$.

Suppose a subset ~~exists~~ exists ~~Then~~ that adds to W. If that subset is the entire set of weights, we explicitly create a yes instance of NPP, if ~~the~~ SUM = 2W, then for ~~if just translating~~ each w_i ~~into an~~ creating an x_i checks if subset of ~~a~~ sum $2W/2 = W$ exists. In the rest of the cases, we add an ~~positive~~ extra positive x_{n+1} to make the total ~~SUM~~ sum in the NPP instance

2W or 2SUM-2W exists, thenus getting an instance that correctly checks if a subset of size W or SUM-W exists. Conversely, if the NPP istance returns yes, then either either SUM = W or because that coresponds to a fixed yes NPP instance; Else, one of S_1 or S_2 contains a subset that adds to W or SUM-W that has elements $\{x_i\}$ that corespond to a subset $\{w_i\}$ of the SS instance (the other contains the extra x_{n+1} hence cannot corespond to a subset of weights).

Since we have shown SS $\leq_p$ NPP and NPP $\in$ NP, we conclude that NPP is NP complete.

28) We can check a solution is correct in polynomial time by taking I as a certificate and checking

28) We can check a solution in polynomial time by taking the set I as a certificate (and

I is polynomial in the size of the instance) and checking that $|I| \geq k$ and there is no path ~~shorter~~ of 2 or fewer edges in G b/w any two nodes in I.

Take any instance of Independent Set (IS). We create an equivalent instance of Strongly Independent Set (SIS) in polynomial time as follows:-
~~for any~~ e if $k < 2$, or $0 \leq k < 2$, create a fixed yes instance of SIS if $|V| \geq k$, else create a fixed no instance. If $k \geq 2$, then ~~for we each (b for each e)~~ for each $(u,v) \in E$, we create an node uv, and we replace edge (u,v) with edges (u, uv) and (uv, v). Addition, we create an edge ~~b/w every pair of the these~~ (uv, uv') for every pair of nodes of this form. We keep the same value of k.

Suppose there exists an independent set of size k. The cases when ~~b~~ $0 \leq k < 2$ are trivial, ~~so suppose~~ if $k \geq 2$ then for every pair of nodes connected by an edge in G, those nodes have a path of 2 edges in the reduced graph G', and ~~for so so cannot~~ those nodes with a shortest path of 2 edges in G have a shortest path of 4 edges in ~~the rest~~ G'.

Suppose there exists an Independent Set I' of size k in G. The case of $k < 2$ is trivial, so assume $k \geq 2$. Then for every pair of nodes u, v in I', u and v do not have an edge b/w them. By our construction, if u and v have a path of 2 edges b/w them, then in the reduced graph G', they have a path of 4 edges b/w them, and so u and v are in the strongly independent set I of G'. so $I' \subseteq I$, and so $|I| \geq k$. Conversely, suppose a strongly independent set I of size at least k exists in G'. Then for any pair of nodes u and v in I, u and v are in G as well because the extra nodes in G' have a path of at most 2 edges to every other node. Additionally, u and v have a shortest path of at least 4 edges in G', and so in G they have a shortest path of at least 2 edges, hence u and v are in an Independent Set I' of G, so $I' \subseteq I$ and so $|I'| \geq k$.

We have shown that SIS $\in$ NP and IS $\leq_p$ SIS, hence SIS is NP complete.

We construct an equivalent instance of Multivariate polynomial minimization in polynomial time as follows.

30) Suppose we have any Y of 3-SAT. Let the 3SAT instance be:-

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

We construct an equivalent instance of MPoM

over boolean variables $X_1, \cdots, X_n$. For each boolean variable X_i we introduce red variable x_i

Consider any literal l. We define $q(l)$ as $1 - x_i$ if $l = X_i$ and x_i if $l = \neg X_i$. Similarly, for clause C_j, we define $Q_j(x) = q(l_{j1}) q(l_{j2}) q(l_{j3})$, where $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$.

Now we define ~~$P(x_1, \cdots, x_n)$~~ our polynomial as:-

$$P(x_1, \cdots, x_n) = \sum_{i=1}^{n} x_i(1 - x_i) + \sum_{j=1}^{m} Q_j(x)$$

We let $B = 0$

has a satisfying assignment.

Suppose ϕ ~~is satisfiable~~. Let $x_i = 1$ if X_i is set to 1, and $x_i = 0$ if $X_i = 0$. Since every x_i is 0 or 1, $\sum_{i=1}^{n} x_i(1 - x_i) = 0$. Also, every clause is satisfied, so each contains one true literal atleast, and hence its associated Q_j has one factor 0 and thus evaluates to 0, and so $P(x_1, \cdots, x_n) = 0 \leq B = 0$ so our multivariable polynomial minimization instance is a 'yes' instance. Conversely,

Suppose there are $x_1, \dots, x_n \in [0,1]$ such that $P(x_1, \dots, x_n) \leq 0$. Every term making up P is non negative, and so $P(x_1, \dots, x_n) \geq 0$. ~~But Since~~ Thus $P(x_1, \dots, x_n) = 0$, and every term in P must evaluate to 0. That is, every $x_i(1-x_i) = 0$ ~~and every $Q_j(x) = 0$~~, so each $x_i \in \{0,1\}$, and every $Q_j(x) = 0$ so at least one literal in every C_j is true thus every clause is satisfied.

Hence, 3SAT $\leq_P$ Multivariable Polynomial Minimization.

32) We can check in polynomial time that a solution is correct by taking ~~the~~ a permutation of the strings in S_δ (which is polynomial in the size of the Perfect Assembly instance, and ~~seeing~~ checking that for each ~~pair~~ $j = 1 \dots n-1$, (s_j, s_{j+1}) is corroborated by some $t_k \in T$. Hence PA $\in$ NP.

(HP) $\propto$ X Let Y be some instance of Hamiltonian Path. We create an equivalent PA instance in polynomial time as follows. For each ~~node~~ $\in$ node $v \in V$ in the HP input graph $G = (V, E)$, assign a ~~char~~ string x_v. Then ~~we put the string $x_v x_v$~~ S is the set of all strings of the form $^9 x_v x_v$ for every node

string x_v, and T is the set of all strings of the form $x_u x_v$ for each edge (u, v) in E.

Suppose Y is a 'yes' instance of HP. Then ∃ a permutation of all nodes of G such that any two consecutive nodes are joined by an edge. Since each pair of consecutive nodes v and u correspond to a pair of strings $x_v x_u$ and $x_u x_v$ in S, and the associated edge (v, u) corresponds to a string $x_v x_u$ in T, it follows that ~~the permutation of strings in S such that~~ a valid permutation of strings in S exists. ~~Conversely, if a valid permutation of strings in~~ so X is also a 'yes' instance of PA. Conversely, if X is a 'yes' instance of PA, the ∃ a ~~valid~~ permutation of strings in S where each consecutive pair of strings is corroborated by some $t_h \in T$, and since each of these pairs ~~is~~ correspond to some ~~node~~ pair of nodes joined by an edge in G, and altogether cover all nodes, a hamiltonian path exists in G so ~~X~~ Y is also a 'yes' instance.

We have shown ~~P~~ HP $\leq_p$ PA and PA $\in$ NP, hence PA is NP complete.

38) ~~He~~ A certifier can take a path P in G as a certificate and check

in polynomial time if the sum of the length of the edges is at most L and the sum of the risks of the edges is of most R. Hence Galactic Shortest Path (GSP) Problem ∈ NP.

We create a GSP instance X in poly time as follows:

Suppose Y is an instance of Subset Sum (SS) with weights $W_1 \cdots W_n$. We create a node v_i for each weight W_i. For each v_i, v_{i+1} we create $e_i = (v_i, v_{i+1})$ and $e_i' = (v_i, v_{i+1})$ And we Let $l_e = W_i$ and $l_{e'} = 0$, similarly $r_e = 0$ and $r_{e'} = W_i$. Let $S = \sum_{i=1}^{n} w_i$. Then we let $L = W$ and $R = S - W$. Let $s = v_1$ and $t = v_n$.

Suppose a If Suppose a subset of the weights add to W. Then the complement of that subset adds to S-W. So suppose such a subset exists. Then for each W_i, we choose e_i, and ge for each W_i not in the subset, we choose e_i', to get an s-t path. This path has length $W = L$ and risk $S - W = R$. Conversely, if a path P meeting the bounds exists, then the set of weights coresponding to the edges e_i in P add to at most W and its complement to at most S-W. But since both sets add up to S,

individually they must add to exactly W and SW. Hence we have a valid SS solution.

Hence GSP ∈ NP and SS ≤P GSP so GSP is NP complete.

39) A certifier can take an instance of Directed Disjoint Paths and as certificate, paths corresponding to each of the pairs provided in the instance and then check in polynomial time that the paths are disjoint, and valid and start and end at the nodes specified by the associated pair. Hence Directed Disjoint Paths (DPP) ∈ NP.

We create an equivalent DPP instance in poly time as follows:

Suppose we have any instance of 3SAT with k clauses and n variables. For each variable x_i, we created vertices s_i and t_i, and we create two paths internally node disjoint paths. For each occurrence of x_i a literal, if it is x_i, place the associated node on one of the s_i, t_i F_i paths and if it is $\bar{x}_i$ then place it on the other T_i. Also we add the pair (s_i, t_i) to the instance. Also for each clause c_i we create two nodes a_j and b_j. Let v_{j1}, v_{j2} and v_{j3} be the three literal occurence nodes already placed on the variable paths. Then we add edges (a_j, v_{jr}) (v_{jr}, b_j) for $r = 1, 2, 3$.

T_i and F_i

and we add (a_j, b_j) to the instance.

Suppose the 3SAT instance is satisfiable. Then for each x_i, if it is true, choose path T_i and if it is false choose path F_i. The n paths are node disjoint because the variable gadgets are seperate. At least one literal in each C_j is true, by our contruction the path $a_j \to v_{ir} \to b_j$ is node disjoint because v_{ir} lies on the unselected variable path. Conversely, if node disjoint paths for each pair exists, then consider the path from s_i to t_i for each variable x_i. If it is using T_i, x_i = true, if it is using F_i, x_i = false. Consider the path a_j to b_j for each clause C_j, the only incoming edges for b_j come from literal vertices of C_j, so the path must go through at least one v_{ir}, and since v_{ir} cannot be on a selected variable path, l_{ir} must be true, so every clause must have at least one true literal, hence the 3SAT instance is satisfiable.

Since DDP $\in$ NP and 3SAT $\leq_p$ DDP, DDP is NP complete

$= (V', E')$ $= (V, E)$

4)a) We create G' from G by creating two nodes v_{in} and v_{out} for each node $v \in V$. Also, for each edge (u, v) in E, we add (u_{out}, v_{in}) to E'. ~~This is~~ G' is a bipartite graph because edges only go from out nodes to in nodes. Now use ~~a flow a~~ check if a bipartite matching exists in G' using a ~~flo~~ network flow algorithm. ~~If it does the~~

perfect

Suppose a bipartite matching exists. Then ~~ex~~ we have found a set of edges such that for each node, exactly one enters and exactly one leaves, and so we have a cycle cover. Conversely, if a cycle cover exists, then each node has exactly one incoming and one outgoing edge in the set of cycles, ~~so~~ and so the coresponding edges form a perfect bipartite matchy in G.

Creating G' takes $O(|V| + |E|)$ time, and the finding a perfect matching ~~takes~~ using Ford Fulkerson takes $O(mn)$ time i.e. polynomial

41)b) A certifier can take as certificate the set of cycles and verify in polynomial time that no two cycles share a node and every node of G is in a cycle. Hence restrict 3 Cycle Cover (3CC) ∈ NP.

Take any 3DM matching instance. We create an equivalent 3CC instance in polynomial time as follows:- For each element in X ∪ Y ∪ Z, create a node and for each triple $t = (x, y, z) \in T$, create nodes a_t, b_t, c_t. For each triple, and add edges (a_t, b_t), (b_t, c_t) and (c_t, a_t). Add bidirectional edges connecting the three new vertices to the coresponding elements.

Suppose a 3DM perfect matching M exists. For each $t \in M$ $t = (a_t, b_t, c_t) \in M$, select the cycles formed to $c_t \to x \to a_t$, $b_t \to y \to b_t$ and $c_t \to y \to c_t$, For the remaining thus covering every element node and every triple triangle. For $t \notin M$, add the cycle $t = (a_t, b_t, c_t) \notin M$, add the cycle $a_t \to b_t \to c_t \to a_t$. This covers the remaining nodes with 3 edge cycles. Conversely, if a per cycle covers with cycles edges at most 3 exists, then only cycles possible are of the form $a_t \to b_t \to c_t \to a_t$ and 2 cycles b/w element nodes and triple nodes. (no 3 edge cycle can contain an element node). If an internal cycle is used 3 edge cycle is used, that triple t is

not in M. Otherwise all three nodes in a triangle form a cycle with an ~~node~~ element node, this triangle's triple is in M. Since this triangle covers three element nodes, and all element nodes are covered, ~~by~~ with node disjoint cycles. the triples in M constit form a perfect 3D matching.

Hence $3DM \leq_P 3CC$ and $3CC \in NP$, thus 3CC is NP complete.